

Unit IV

File Organization and Disk scheduling Algorithms

File Concept

A file is a named Collection of related information

- ✧ It is the logical storage unit.
 - ✧ Commonly file contains programs and data.
 - ✧ Information about files are kept in the **directory structure**, which is maintained on the disk
 - ✧ OS maps files to physical devices.
 - ✧ Types of files
 - ✧ Data
 - ✧ Program
 - ✧ Documents
-

File Attributes

- ❖ **Name** - symbolic file-name, in human-readable form
 - ❖ **Identifier** - This unique tag, it is the non-human-readable name for the file
 - ❖ **Type** - for systems that support multiple types
 - ❖ **Location** - pointer to a device and to file location on device
 - ❖ **Size** - current size of the file
 - ❖ **Protection** - controls who can read, write, execute
 - ❖ **Time, Date and user identification**
 - data for protection, security and usage monitoring
-

File Operations

❖ A file is an abstract data type. The different operations that can be performed are:

❖ Create

❖ Write

❖ Read

❖ Reposition - file seek

❖ Delete

❖ Truncate

File Organization - Access Methods

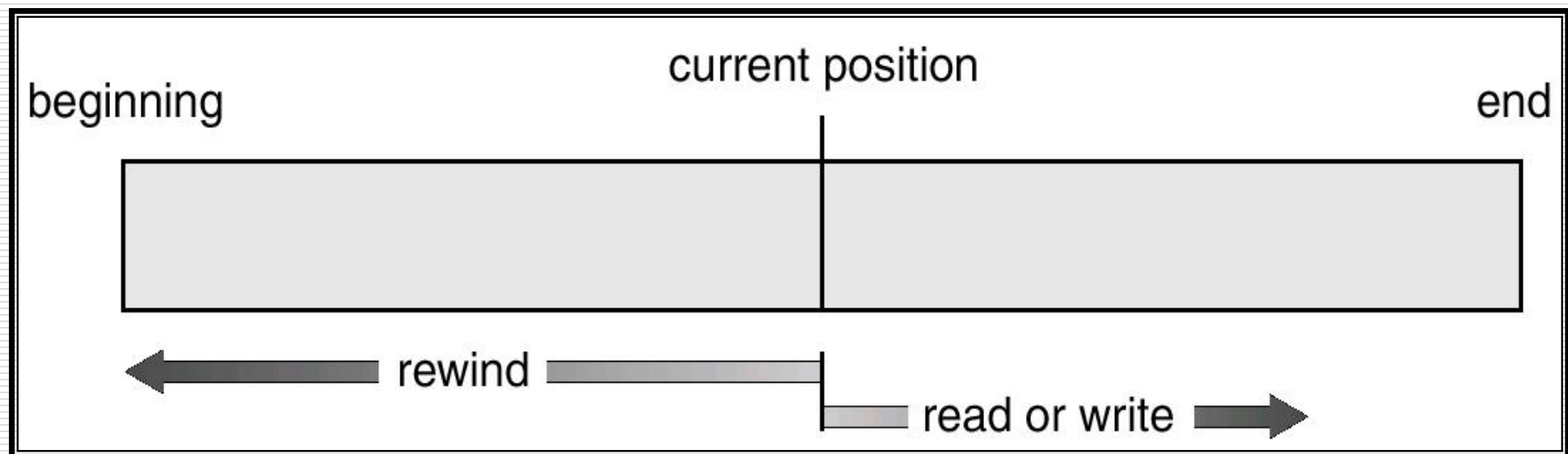
❖ A file can be accessed in several ways

❖ **Sequential Access**

❖ **Indexed Access**

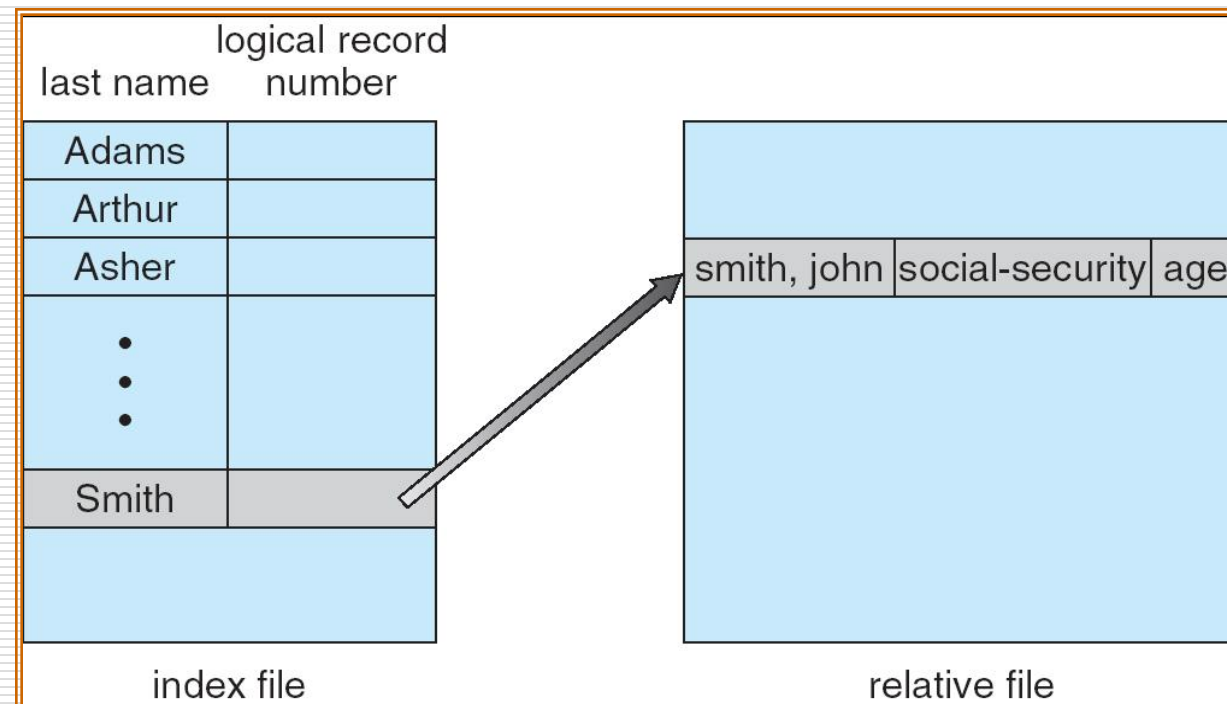
Sequential-access File

- ❖ The simplest access method is sequential access.
- ❖ Here the information in the file is processed in order.
- ❖ A read operation reads the next portion of the file and advances the file pointer.
- ❖ Write operation appends at the end of the file.



Indexed Accessing

- ❖ Here we construct an index for the file .
- ❖ The index contains pointer to the various blocks.
- ❖ To find a record in the file we first search the index and then use the pointer to access the file.



Directory Structure

- ❖ To manage all the data on the disk we need to organize them.
- ❖ This organization is done in two parts.
 - ❖ First the disk is divided into one or more **partitions**.
 - ❖ Each partition is divided into one or more **directories**.

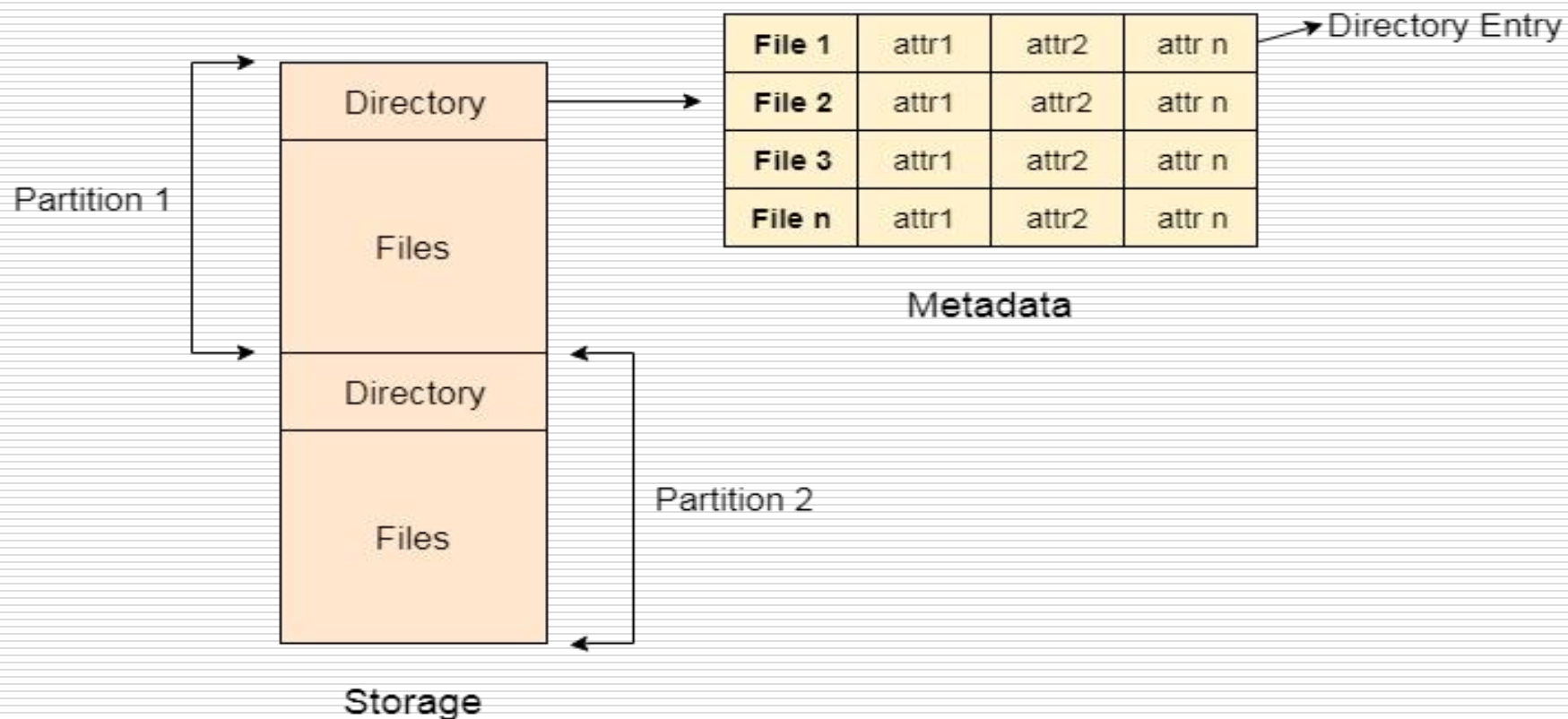
Device Directory:

A collection of nodes containing information about all files on a partition.

Directory Structure

Directory can be defined as the listing of the related files on the disk. The directory may store some or the entire file attributes.

Each partition must have at least one directory in which, all the files of the partition can be listed. A directory entry is maintained for each file in the directory which stores all the information related to that file.



Information in a Device Directory

- ❖ File Name
 - ❖ File Type
 - ❖ Address or Location
 - ❖ Current Length
 - ❖ Maximum Length
 - ❖ Date created, Date last accessed
 - ❖ Date last updated
-

Operations Performed on Directory

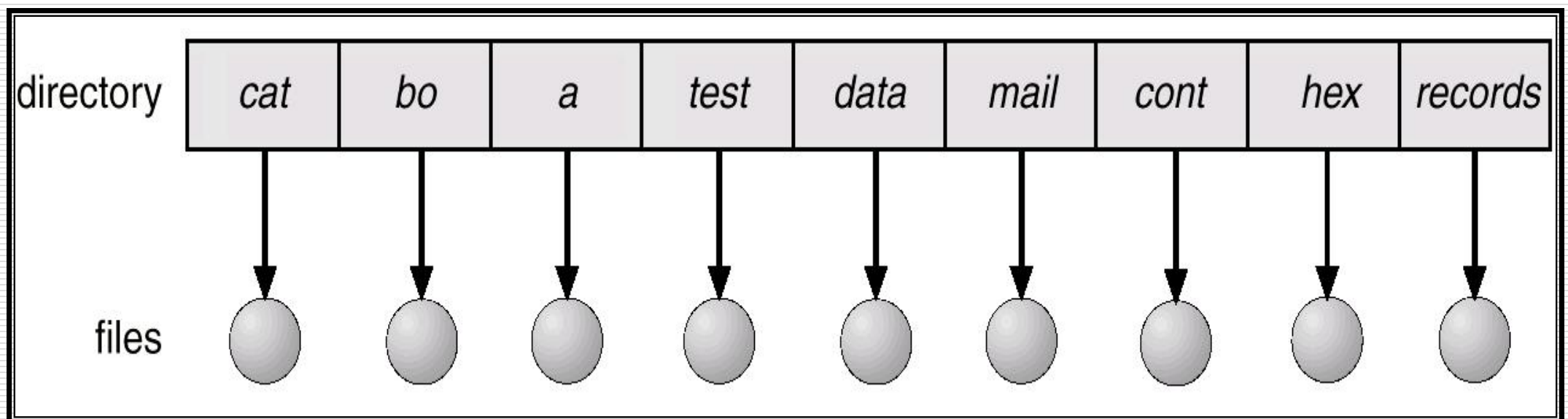
- ❖ Search for a file
 - ❖ Create a file
 - ❖ Delete a file
 - ❖ List a directory
 - ❖ Rename a file
 - ❖ Traverse the file system
-

Operations Performed on Directory

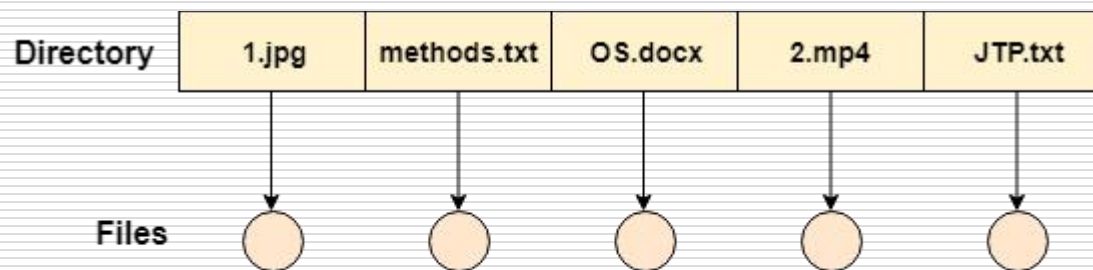
- ❑ **Search for a file:** We need to be able to search a directory structure to find the entry for a particular file.
 - ❑ **Create a file:** New files need to be created and added to the directory.
 - ❑ **Delete a file:** When a file is no longer needed, we want to remove it from the directory.
 - ❑ **List a directory:** We need to be able to list the files in a directory, and the contents of the directory entry for each file in the list.
 - ❑ **Rename a file:** Name of a file represents its contents to its users, the name must be changeable when the contents or use of the file changes.
 - ❑ **Traverse the file system:** We may wish to access every directory, and every file within a directory structure.
-

Single Level Directory

- ❖ A single directory for all users
- ❖ Naming Problem and Grouping Problem
 - ❖ As the number of files increases, difficult to remember unique names
 - ❖ As the number of users increase, users must have unique names.



Single Level Directory



Single Level Directory

The simplest method is to have one big list of all the files on the disk. The entire system will contain only one directory which is supposed to mention all the files present in the file system. The directory contains one entry per each file present on the file system.

Single Level Directory

□ Advantages

1. Implementation is very simple.
2. If the sizes of the files are very small then the searching becomes faster.
3. File creation, searching, deletion is very simple since we have only one directory.

□ Disadvantages

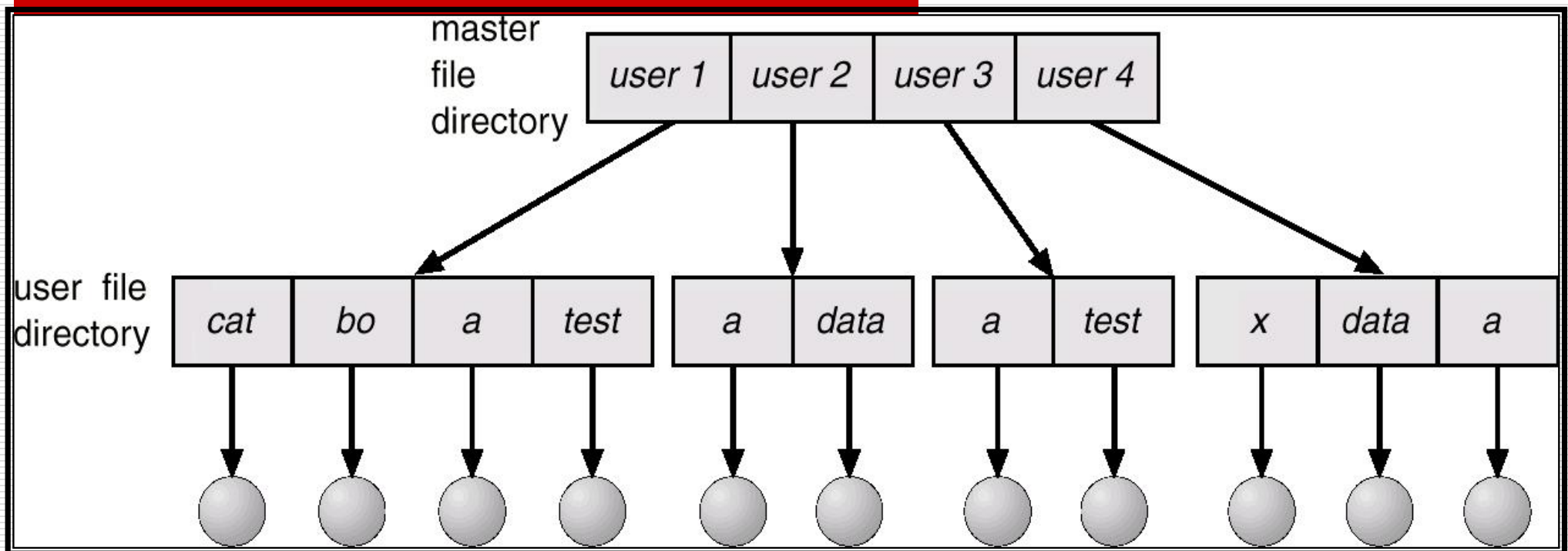
1. We cannot have two files with the same name.
 2. The directory may be very big therefore searching for a file may take so much time.
 3. Protection cannot be implemented for multiple users.
 4. There are no ways to group same kind of files.
 5. Choosing the unique name for every file is a bit complex and limits the number of files in the system because most of the Operating System limits the number of characters used to construct the file name.
-

Two Level Directory

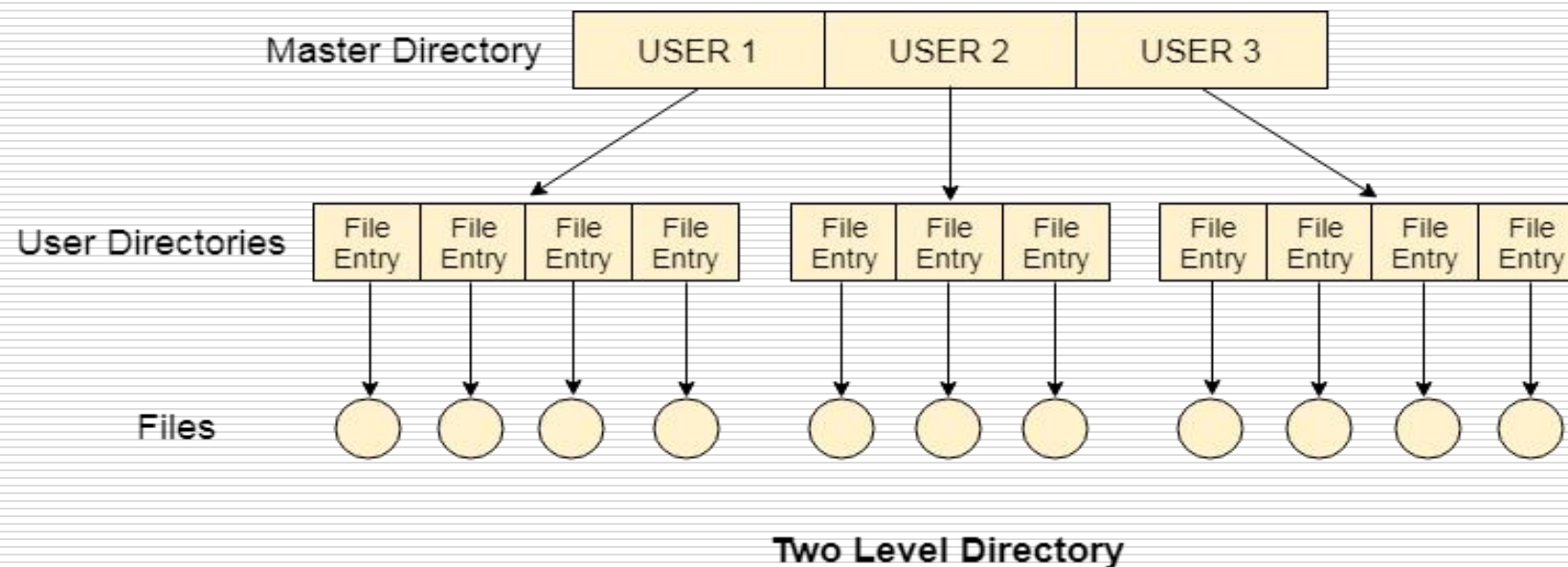
Introduced to remove naming problem between users

- ❖ First Level contains list of user directories
 - ❖ Second Level contains user files
 - ❖ Need to specify Path name
 - ❖ Can have same file names for different users.
 - ❖ System files kept in separate directory or Level 1.
 - ❖ Efficient searching
-

Two Level Directory



Two Level Directory



In two level directory systems, we can create a separate directory for each user. There is one master directory which contains separate directories dedicated to each user. For each user, there is a different directory present at the second level, containing group of user's file. The system doesn't let a user to enter in the other user's directory without permission.

Two Level Directory

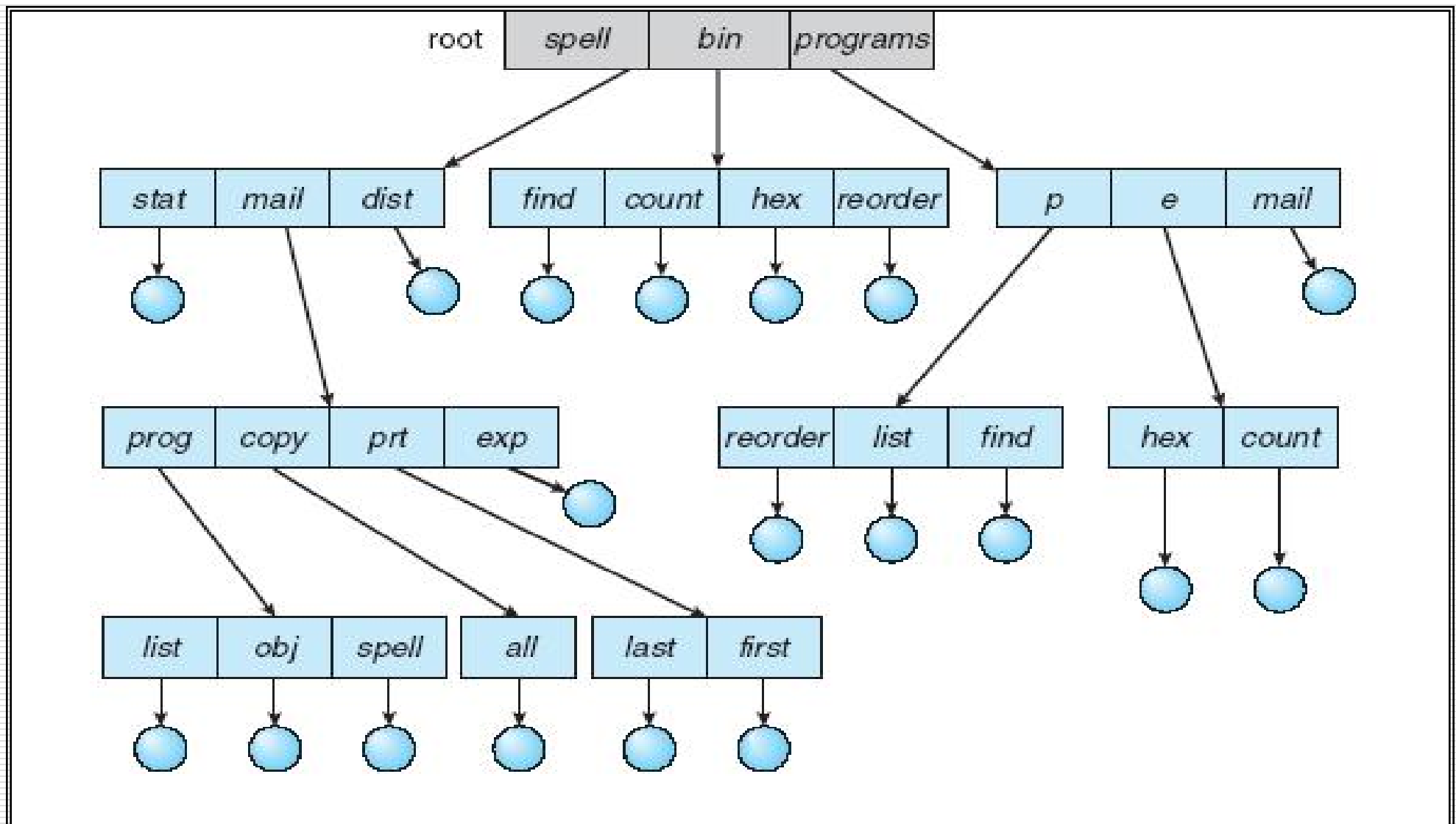
Characteristics of two level directory system

1. Each file has a path name as */User-name/directory-name/*
 1. Different users can have the same file name.
 2. Searching becomes more efficient as only one user's list needs to be traversed.
 3. The same kind of files cannot be grouped into a single directory for a particular user.
-

Tree Structured Directories

- ❑ Arbitrary depth of directories
 - ❑ Leaf nodes are files, interior nodes are directories.
 - ❑ Efficient Searching
 - ❑ Grouping Capability
 - ❑ Current Directory (working directory)
 - ❑ MS-DOS, Linux uses a tree structured directory
-

Tree structured Directories



Tree Structured Directories

- ❖ Absolute or relative path name
 - ❖ Absolute from root
 - ❖ Relative paths from current working directory pointer.
 - ❖ Creating a new file is done in current directory
 - ❖ Creating a new subdirectory is done in current directory, e.g. `mkdir <dir-name>`
 - ❖ Delete a file , e.g. `rm file-name`
 - ❖ Deletion of directory
 - ❖ Option 1 : Only delete if directory is empty
 - ❖ Option 2: delete all files and subdirectories under directory
-

Allocation Methods

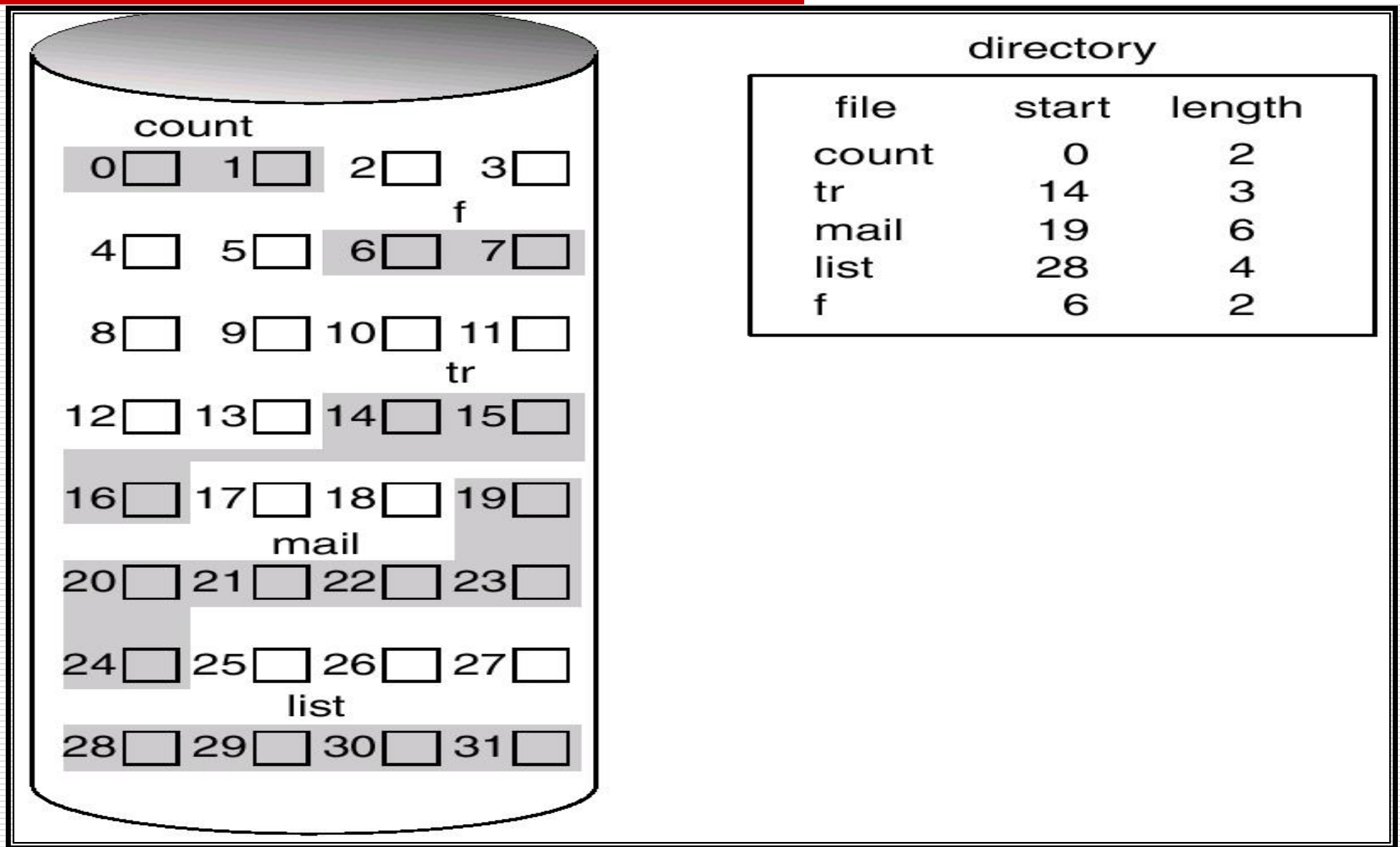
An allocation method refers to how disk blocks are allocated for files.

- ❖ Contiguous Allocation
 - ❖ Linked Allocation
 - ❖ Indexed Allocation
-

Contiguous Allocation

- ❖ Each file occupies a set of contiguous blocks on the disk.
 - ❖ Simple - only starting address and length are required.
 - ❖ Suitable for sequential or direct access.
 - ❖ Fast (very little head movement) and easy to recover if system crashes.
 - ❖ Problems
 - ❖ Wastage of disk space.
 - ❖ Use first fit or best fit. Leads to external fragmentation on disk.
 - ❖ Files cannot grow - expanding file requires copying
-

Contiguous Allocation



Linked Allocation

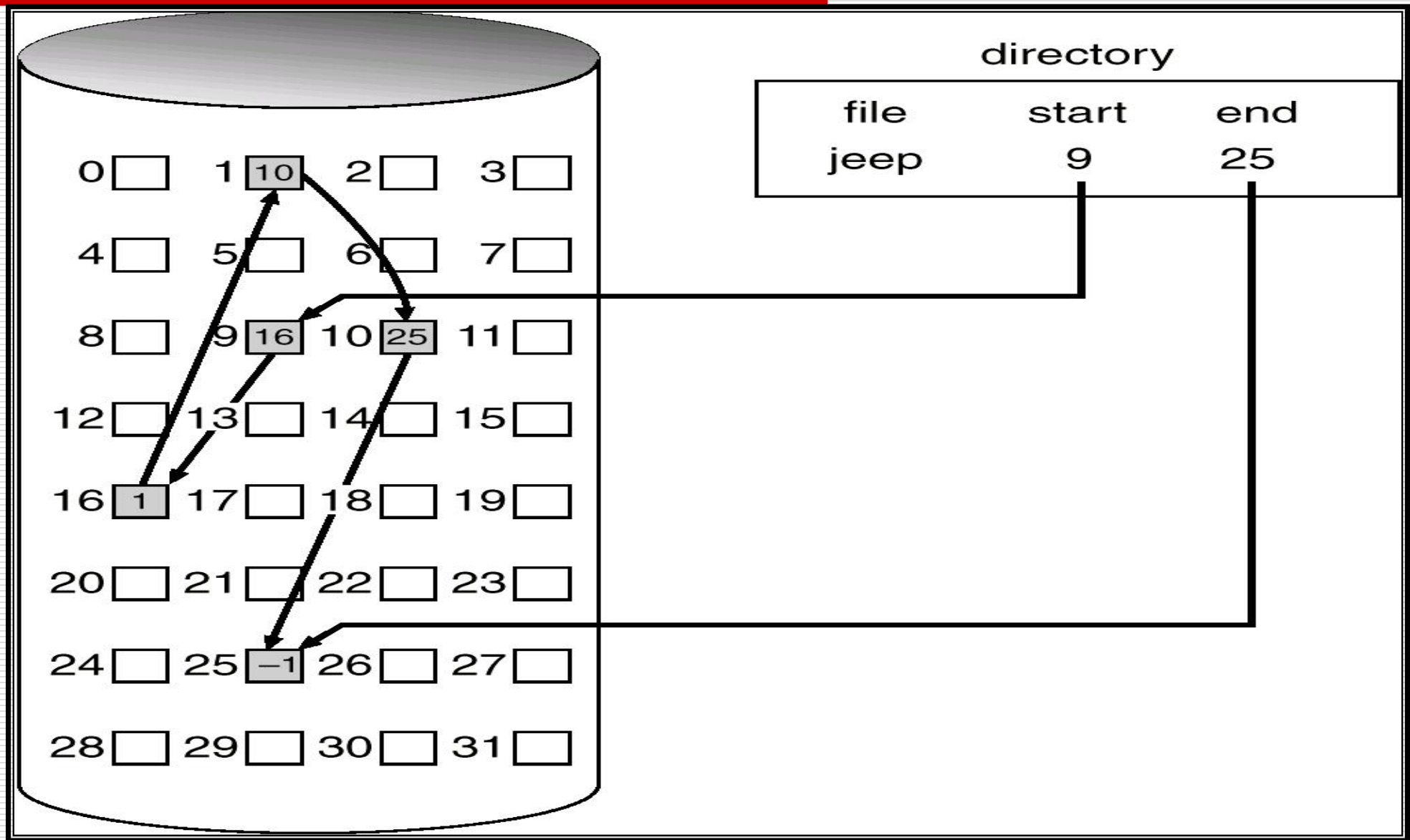
Each file is a linked list of disk blocks

- ❖ Blocks may be scattered anywhere on the disk.
- ❖ The directory contains a pointer to the first and last blocks of a file.
- ❖ Each block contains a pointer to the next block.

❖ Disadvantages

- ❖ Lot of disk space used to store pointers,
 - ❖ Efficient for sequential access files .
-

Linked Allocation

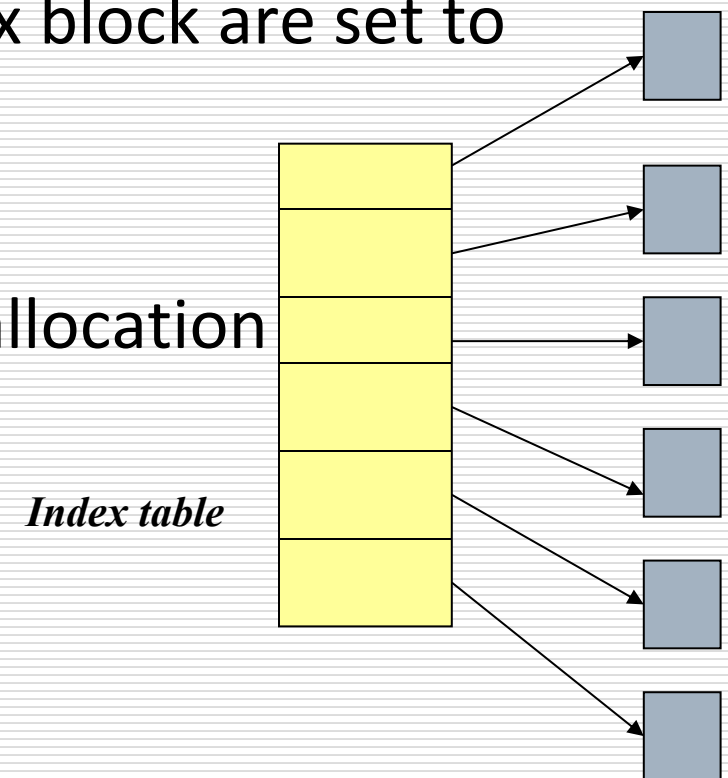


Indexed Allocation

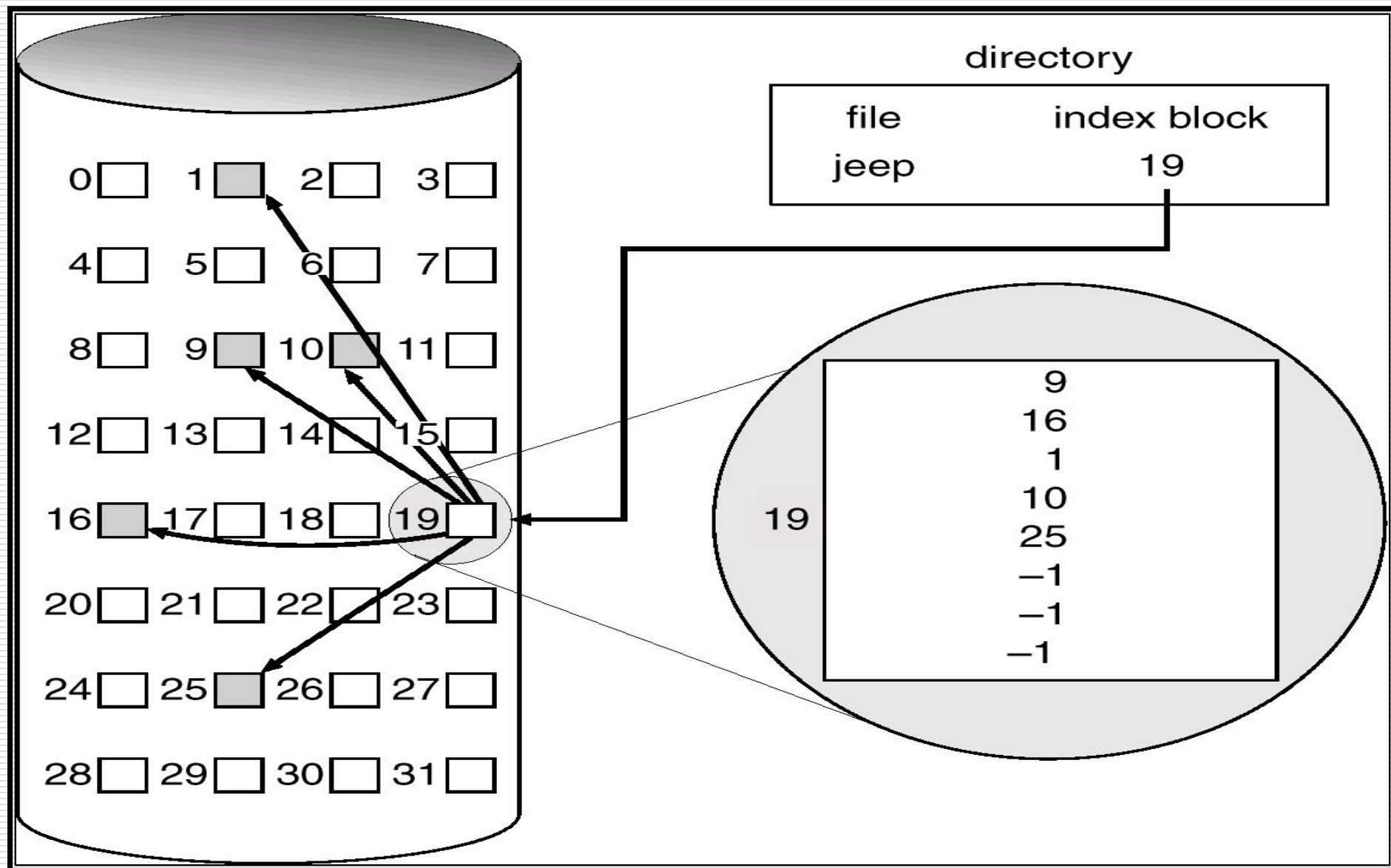
- ❖ All pointers are stored into one location - **The index block.**
- ❖ Need index table.
- ❖ Supports sequential, direct and indexed access.
- ❖ For a new file all the pointers in the index block are set to NILL

Disadvantage

- ❖ pointer overhead – Greater than linked allocation



Indexed Allocation



Disk Structure

- ❖ Disks are used as secondary storage medium for modern computer.
- ❖ Disks are addressed as a large one dimensional arrays of logical blocks.
- ❖ The size of the logical block is 512 bytes.
- ❖ The logical blocks is mapped onto the sectors of the disk sequentially.
 - ❖ Sector 0 is the first sector of the first track on the outermost cylinder.

Disk Scheduling

- ❖ The number of sectors per track has been increasing as disk technology improves
- ❖ When a process needs an I/O to or from the disk, it issues a system call to the OS .
- ❖ The request specifies several pieces of information such as:
 - ❖ Whether this operation is input or output
 - ❖ The disk address for the transfer
 - ❖ The memory address for the transfer
 - ❖ The number of sectors to be transferred

Disk Scheduling

- ❖ If the desired disk drive and controller are available the request can be processed immediately.
- ❖ If it is busy new requests for service will be placed in the queue of pending requests for that drive.
- ❖ For multiprogramming system the disk queue may have several pending requests.
- ❖ scheduling algorithms decide which pending request to service next

Disk Scheduling-terms

Access time

- ❖ Time taken total time taken to locate the sector to read or write

Seek time

- ❖ It is the time for the disk head to travel to the track of the disk where the data will be read or written.

Rotational latency

- ❖ It is the additional time waiting for the disk to rotate the desired sector to the disk head.

Disk Scheduling

Several algorithms exist to schedule the servicing of disk I/O requests.

- ❖ FCFS
- ❖ SSTF
- ❖ SCAN
- ❖ C-SCAN

FCFS

- ❖ The simplest form of disk scheduling is the FCFS scheduling
- ❖ This algorithm is fair.
- ❖ But it generally does not provide fastest service.

FCFS

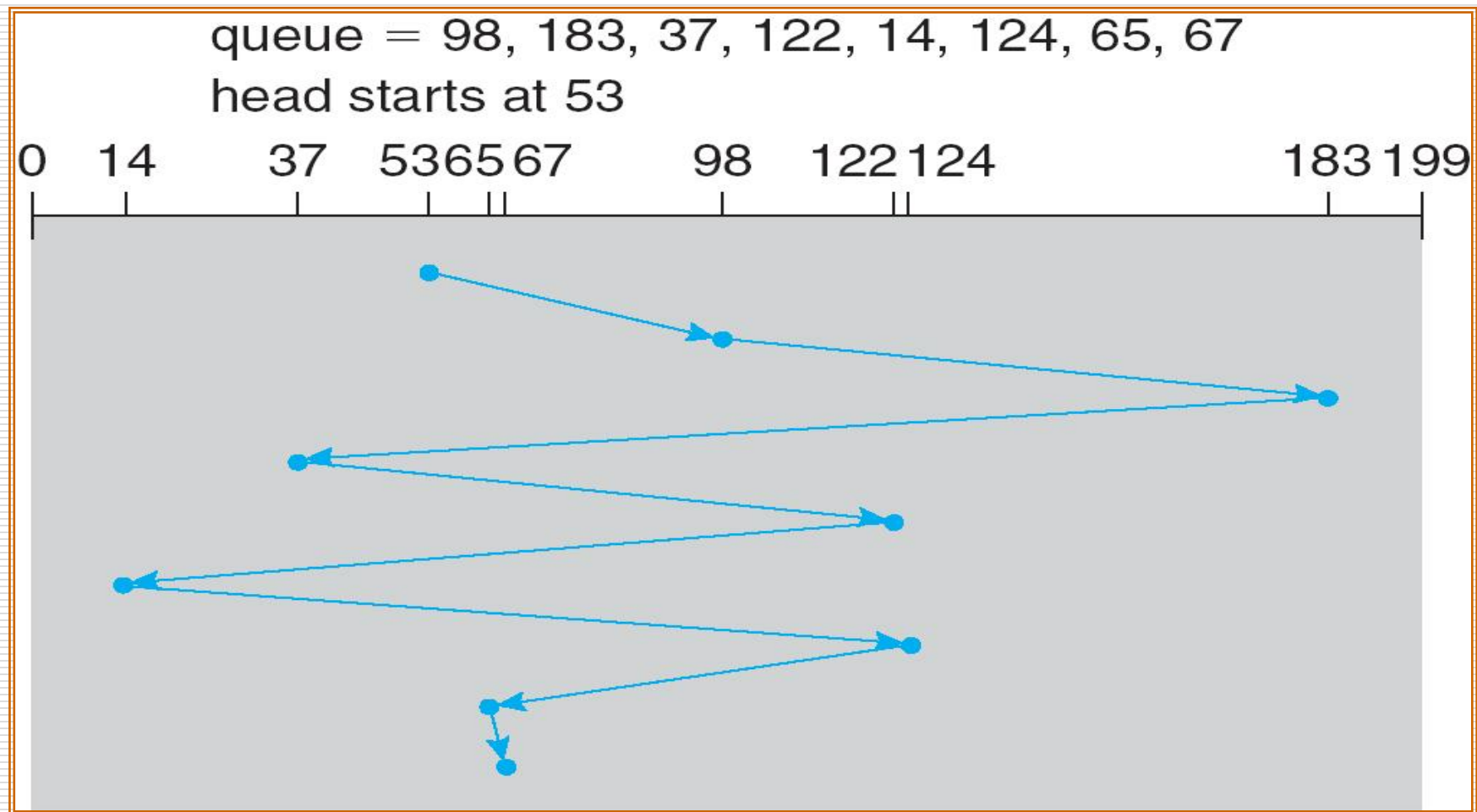


Illustration shows total head movement of 640 cylinders.

SSTF

- ❖ Selects the request with the minimum seek time from the current head position.
- ❖ The seek time increases with the number of cylinders traversed by the head ,it chooses the pending request closest to the current head position.
- ❖ SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.

SSTF

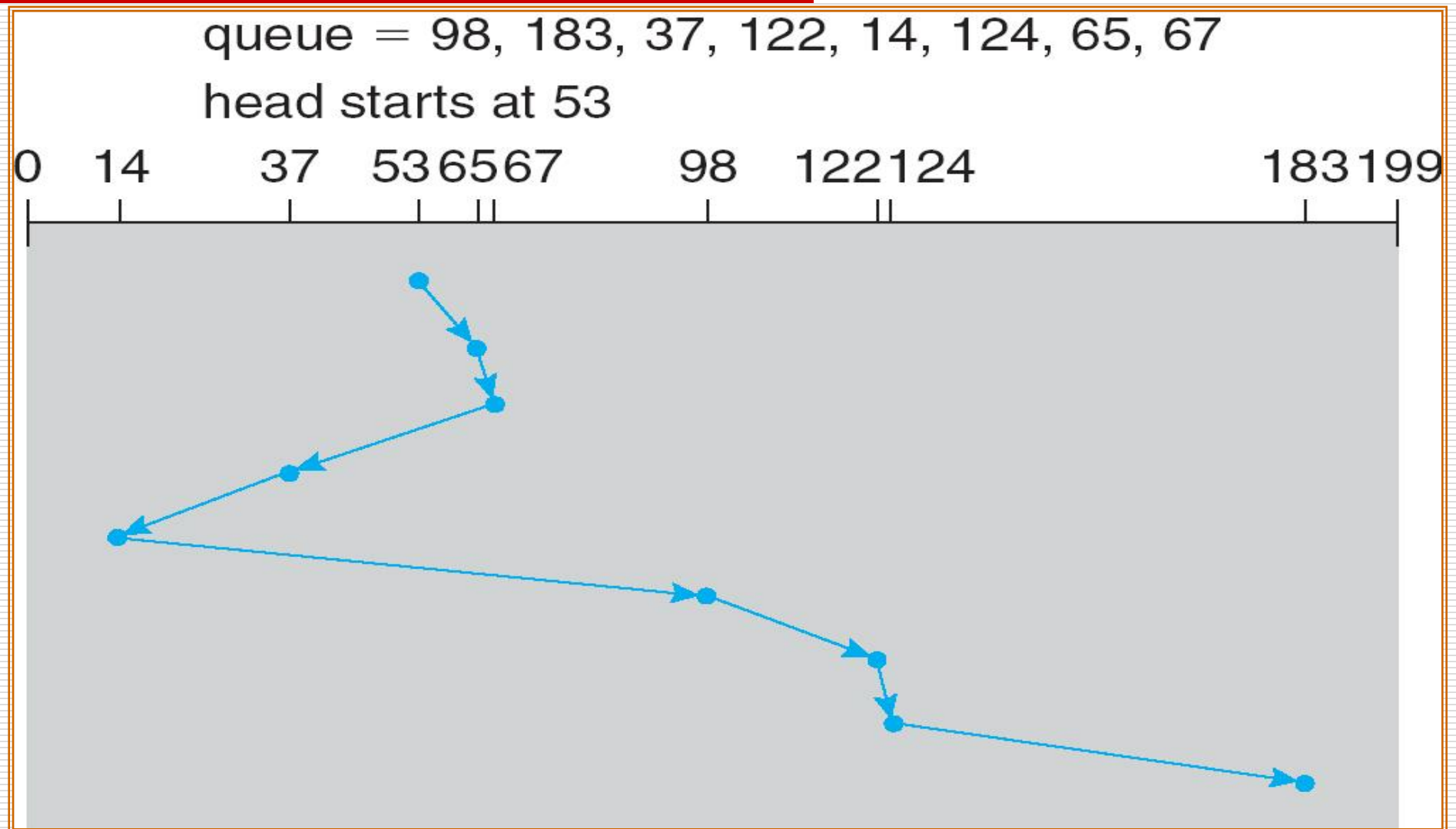


Illustration shows total head movement of 236 cylinders.

SCAN

- ❖ The disk arm starts at one end of the disk, and moves toward the other end
- ❖ Requests are serviced as it reaches each cylinder until it gets to the other end of the disk.
- ❖ At the other end the direction of head movement is reversed and servicing continues.
- ❖ The head continuously scans back and forth across the disk.
- ❖ Also called the *elevator algorithm*.

SCAN (Cont.)

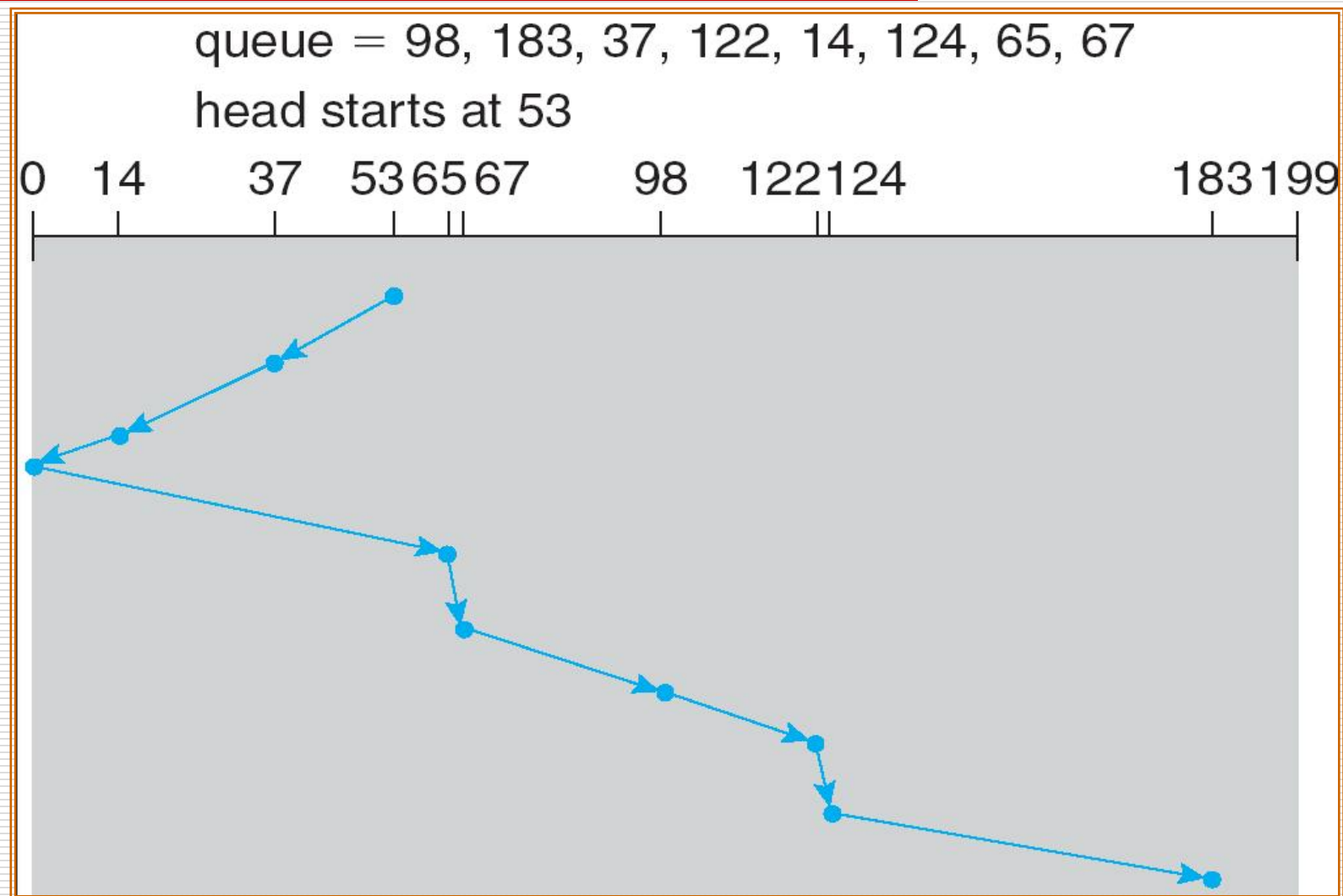


Illustration shows total head movement of 208 cylinders.

C-SCAN

- ❖ It is a variant of SCAN scheduling.
- ❖ Provides a more uniform wait time than SCAN.
- ❖ CSCAN moves the head from one end of the disk to the other.
- ❖ servicing requests along the way. When head reaches the other end, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- ❖ CSCAN scheduling algorithm treats the cylinders as a circular list that wraps around from the last cylinder to the first one.

C-SCAN

